SOFTWARE CONSTRUCTION

Software Construction & Development

Lectures 4

OVERVIEW

Specifications

The specific topics covered in this module are listed below, and are the basis for the outline of its content.

- A Construction Planning
- B Code Design
- C Data Design and Management
- D Error Processing
- E Source Code Organization
- F Code Documentation
- G Construction Quality Assurance
- H System Integration and Deployment
- I Code Tuning
- J Construction Tools

F

Code Documentation

Why It Matters

Good documentation shows professionalism and makes software easier to use, maintain, and improve.

1. INTERNAL DOCUMENTATION (WITHIN THE CODE)

- **Commented Code:** Clarifies logic, variables, and purpose.
- **Inline Documentation:** Explains complex sections for future developers.

EXAMPLE

```
# Calculates area of a circle given
radius
def calculate_area(radius):
    return 3.14 * radius * radius
```

Types of Documentation (Cont.)

2. EXTERNAL DOCUMENTATION (SEPARATE FROM THE CODE)

- **User Manuals:** For end users to understand features.
- **Operator Manuals:** Instructions for system admins/operators.
- **Maintenance Manuals:** For developers fixing or upgrading the system.
- **Design Documents (SDD):** Describe architecture and design.
- **Unit Development Folder (UDF):** Contains all docs for a specific module (design, code, test results, issues, etc.).

Detailed Design Documents Include

- **Identification:** Name of component
- **Purpose:** What it does
- **Function:** Its key functionality
- **Dependencies:** What other parts it relies on
- **Interface:** How it connects with other parts
- **Resources:** Files, databases, APIs it uses
- **Data:** Internal data it handles

Detailed Design Example — “Add to Cart”

NAME	AddToCart
TYPE	Function
PURPOSE	Adds a selected item to the user's shopping cart
FUNCTION	Takes item ID and quantity, stores in user's cart
INTERFACE	addToCart(itemId, quantity)
DATA USED	Cart, Item (product info), UserID

G

Construction Quality Assurance

What is Construction QA?

Ensures the code meets standards, is reliable, and performs well.

QUALITY ATTRIBUTES

- **Correctness:** Does it do what it should?
- **Reliability:** Can it run without crashing?
- **Maintainability:** Can others easily update or fix it?
- **Portability:** Can it run on different systems?
- **Reusability:** Can the code be reused elsewhere?
- **Readability:** Is the code easy to read?
- **Verifiability:** Can its outputs be tested?

EXAMPLE

If your code uses standard libraries and follows naming conventions, it's more maintainable and readable.

How to Improve Code Quality

To improve code quality, focus on writing clear, maintainable, and well-tested code. Key strategies include:

1

READABLE CODE

Use meaningful names & keep functions short. Follow consistent formatting. Add helpful comments and documentation.

2

TESTING

Automate testing. Test regularly throughout development — apply testing at different levels (unit, integration, system).

3

CODE REVIEWS

Conduct frequent reviews. Prioritize clarity and maintainability. Give constructive feedback.

Refactoring & Collaboration

4. REFACTORING

- Continuously improve code structure.
- Address technical debt.

5. COLLABORATION & CONTINUOUS INTEGRATION

- Share knowledge across the team.
- Use tools to automate builds and tests.

H

System Integration and Deployment

System Integration

- The act of merging a software element or elements with another element.
- The act of merging a hardware component or components with another hardware component.
- The act of merging software configuration items with hardware configuration items in order to produce a total system which satisfies customer requirements.

TYPES OF INTEGRATION TECHNIQUES

- All-in-one integration (a.k.a. the “big bang” integration)
- Incremental integration
- Top-down integration
- Bottom-up integration

Integration Techniques

1

BIG BANG INTEGRATION

Everything is integrated at once. High risk, hard to trace issues — like flipping all switches at once and hoping it works.

2

INCREMENTAL INTEGRATION

Recommended approach: build and test small parts, gradually add more components. Easier debugging and better progress tracking.

3

TOP-DOWN INTEGRATION

Start with higher-level functions, add lower ones. Uses stubs to simulate lower modules.

4

BOTTOM-UP INTEGRATION

Start with the lowest-level components and move upward toward the complete system.

Deployment

A

ALPHA TESTING

Internal testing within the development team.

B

BETA TESTING

Limited release to a group of external users.

L

LIVE DEPLOYMENT

Full release to customers.

EXAMPLE

A mobile app is beta-tested by 100 users before release to the app store.

I

Code Tuning

Code Tuning

- Code tuning is the practice of modifying correct code to make it run more efficiently.
- Improving code performance without changing what it does.

> ***BUT*** *Never optimize too early. First make it work, then make it fast.*

STEPS

1. **Measure:** Identify slow parts (bottlenecks).
2. **Analyze:** Find the cause — bad algorithm? Too many I/O operations?
3. **Optimize:** Rewrite only the necessary part.
4. **Test again:** Make sure it's still correct.
5. **Repeat:** As needed.

Key Points & Common Problem Areas

KEY POINTS

- **Focus on bottlenecks:** Identify slow parts of code using profiling tools (find “hot spots”).
- **Small, local changes:** Optimize loops, expressions, and data structures.
- **Efficiency goal:** Reduce execution time, memory use, or resource consumption.
- **Iterative:** Repeatedly test and refine changes.

COMMON PROBLEM AREAS

- Slow loops
- Unnecessary file/database operations
- Poor algorithms (e.g., using a linear search where a binary search is better)

J

Construction Tools

Why Use Construction Tools?

Construction tools are used to improve productivity and software quality.

WRITE CLEAN CODE

Structured, consistent tooling helps enforce good habits.

SAVE TIME

Automate repetitive tasks so developers focus on solving problems.

REDUCE BUGS

Catch issues early with analyzers, linters, and tests.

WORK BETTER IN TEAMS

Shared tooling keeps everyone aligned and in sync.

Tools for Writing & Testing Code

1. CODE WRITING TOOLS

Code Editors
Where developers write code — e.g. VS Code, Atom.

Auto-formatters
Make code neat and consistent — e.g. Prettier, Black.

Snippets/Templates
Reusable code blocks — e.g. VS Code snippets.

2. CODE QUALITY TOOLS

Static Code Analyzers
Check code quality & complexity — e.g. SonarQube, CodeClimate.

Refactoring Tools
Help clean and simplify code — e.g. Eclipse.

3. DEBUGGING & TESTING TOOLS

Debuggers
Find and fix bugs — e.g. Chrome DevTools, PyCharm Debugger.

Unit Testing Tools
Verify code works — e.g. JUnit, pytest, Jest.

Profilers
Show which parts of code are slow — e.g. VisualVM, cProfile.

Design, Planning & Collaboration Tools

4. DESIGN & PLANNING TOOLS

- **Diagram Tools:** Help plan software visually — e.g. Lucidchart, Draw.io, Figma.
- **ER Diagram Tools:** Design databases — e.g. DrawSQL, Creately, dbdiagram.io.

5. COLLABORATION & VERSION CONTROL

- **Version Control:** Track code changes — e.g. Git + GitHub.
- **CI/CD Tools:** Automate testing and deployment — e.g. GitHub Actions, Jenkins.

Thank You

Software Construction & Development — Lectures 4